

מטלת סיכום
יסודות מתמטיים ללמידת מכונה
יגל אלפסי 208788000

1) Assume you have trained a logistic regression model. Explain what its ROC curve is and how exactly is it computed. What is an 'optimal' ROC curve?

עקומת ROC הינה עקומה שמטרתה לתת ויזואליזציה של הערכת ביצועי המודל הבינארי - מודל המסווג ל-2 קטגוריות כדוגמת במודל רגרסיה לוגיסטית.

ברגרסיה הלוגיסטית אנחנו מקבלים כפלט הסתברות להשתייך לאחת מ-2 הקטגוריות המסומנות ב-0 או 1. לכן ניתן לחשב את המדדים הבאים לפי נקודות סף (Threshold)

$$FPR = \frac{FP}{FP + TN}$$

$$TPR = \frac{TP}{TP + FN}$$

מדדים אלו מהווים את ציר ה-x ואת ציר ה-y בהתאמה של גרף ה-ROC. הגרף נבנה לפי בחירת נקודות סף שונות וערכי ה-FPR ו-TPR השונים שהתקבלו במודל.

גרף ROC אופטימלי הינו כזה אשר מגיע לפינה השמאלית העליונה - כך שערך ה-FPR נמוך וערך ה-TPR גבוה (0,1) - זה מודל מושלם אשר יודע לסווג בצורה נכונה את כלל הדגימות שקיבל.

עקומת ROC מאפשרת כך גם להשוות בין מודלים שונים ע"י השוואת השטח שמתחת לגרף של כל מודל שמתקבל (נקרא גם (AUC

כאשר מודל מושלם קרוב ל-AUC=1 ומודל אקראי ל-AUC=0.5.

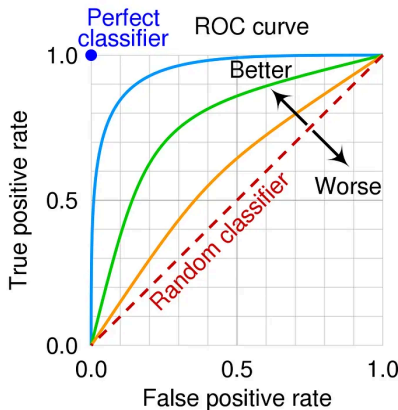


Figure 1: ROC לעקומת

2) For any $N \geq 1$, find an optimal adaptive N-term piecewise constant approximation to $f(x) = x^2$ in $L_\infty[0, 1]$.

ראינו בהרצאות כי נרצה למצוא את הקירוב הטוב ביותר לקבוצה Σ_N .
 בנוסף עבור $f(x) = x^2$ מתקיים $f'(x) = 2x$ בכל הישר הממשי ובפרט ב $[0,1]$.
 נשתמש באותה חלוקה שראינו בהרצאה המקיימת:

$$\int_{t_j}^{t_{j+1}} |f'| \leq \frac{\|f'\|_1}{N} \underbrace{\quad}_{\text{במקרה שלנו}} \frac{1}{N}$$

מתקיים:

$$\begin{aligned} \int_0^1 f' dx &= \int_0^1 2x dx = 1 \\ \Rightarrow \int_{t_j}^{t_{j+1}} f'(u) du &= \int_{t_j}^{t_{j+1}} 2u du = \frac{1}{N} \Rightarrow [x^2]_{t_j}^{t_{j+1}} = t_{j+1}^2 - t_j^2 = \frac{1}{N} \\ t_j &= \left(\frac{j}{N}\right)^{0.5} \end{aligned}$$

עבור $t_0 = 0$ מתקיים $t_1 = \frac{1}{\sqrt{N}}$ נראה באינדוקציה כי מתקיים
 עבור מקרה הבסיס הראנו כבר, כעת נניח כי עבור כל j מתקיים $t_j = \left(\frac{j}{N}\right)^{0.5}$ נראה שגם עבור $j+1$.

$$t_j = \left(\frac{j}{N}\right)^{0.5} \text{ מהנחת האינדוקציה}$$

ולכן:

$$t_{j+1}^2 - \frac{j}{N} = \frac{1}{N} \Rightarrow t_{j+1} = \left(\frac{j+1}{N}\right)^{0.5}$$

כנדרש,

ראינו בהרצאה כי בחירת החציון בכל קטע $[t_j, t_{j+1}]$ לפי ציר ה y ונסמנו M_j מתקיים כי:

$$|x^2 - M_j| \leq \frac{\frac{j+1}{N} - \frac{j}{N}}{2} = \frac{1}{2N}, \forall x \in [t_j, t_{j+1}]$$

כלומר לכל קטע ולכול x בקטע, אם נחסר ממנו את החציון הוא יהיה קטן מאמצע הקטע ולכן אם נבחר את הפונקציה

$$g = \sum_{j=0}^{N-1} M_j 1_{[t_j, t_{N-1}]}$$

$$\|f - g\|_\infty \leq \frac{1}{2N}$$

אזי, עבור $N \rightarrow \infty$ השגיאה תשאף ל-0. בנוסף ניתן לראות כי $g \in L_\infty[0, 1]$ פונקציה חסומה בקטע $[0,1]$.

3) We have a classification dataset with n features and L classes.

- Explain in detail how to evaluate feature importance using the wavelet method.
- Explain how to use an evaluation dataset (not used during training), to ensure the feature importance method is less susceptible to noise.

סעיף א':

ראשית, על מנת לחשב את ה Feature importance (FI) (נסמנו FI) באמצעות wavelet, עוברים על העצים ב RF וסוכמים את נורמות ה wavelets שנוצרו על ידי חלוקה כל פעם ב feature אחר. ראינו בהרצאה כי ה wavelet "מכמת" לנו כמה מכל חלוקה אנחנו מרווחים "מידע" ולכן לכל סכימה נקבל כמה "מידע" קיבלנו בחלוקה לפי feature מסויים. לכל feature $1 \leq i \leq n$ נחשב: עבור J מספר העצים

$$S_i^T = \frac{1}{J} \sum_{j=1}^J \sum_{\Omega \in T_i \cap V_i} \|\psi\|_2^T$$

כאשר $\tau > 0$, V_i קבוצת תת-חלוקות שנוצרות על ידי חלוקה ע"י המאפיין ה- i (בתוך הקבוצה ה- Ω). נסדר S_i^T לפי הגודל וכך אפשר לדעת את החשיבות היחסית של ה feature - כמה "מידע" חלוקה זו נקבל.

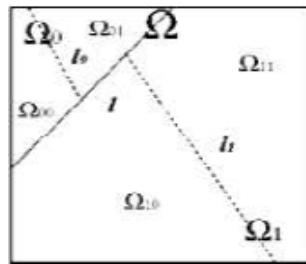


Figure 2: דוגמה מההרצאה לחלוקה בעץ החלטה בודד

סעיף ב':

השיטה הקלאסית להורדת רגישות לרעש בדאטא היא על ידי בחירת נקודת סף גבוהה של "המידע" המתקבל מכל סכימת wavelet. עבור δ א"ש כלשהי נבחר את ה wavelets שנותנים ערך גבוה מ- δ . דרך נוספת היא שימוש בתוצאות ה FI שנקבל מ-2 הסטים - אם נקבל כי המודל מתאים לשניהם את אותם features או לפחות רשימת features דומה, (לדוגמה נגיד ונרצה את ה 10 הראשונים ובסט האימון ה feature הראשון מופיע במקום ה 3 ושאר features "זזו" 2 מקומות לכל כיוון במקסימום בסט הערכה ובנוסף מתוך ה 10 בסט האימון 8 מופיעים בסט הערכה). נגיד כי המודל לא רגיש לרעש, אם נקבל רשימת features שונה לחלוטין או בצורה ניכרת נגיד שהמודל רגיש לרעש או overfit.

4) Prove that for any $\alpha < \frac{1}{\tau}$, there exists a constant $c(\alpha, \tau) > 0$, such that

$$|\Delta_j|_{B_\tau^\alpha[0,1]} \leq c 2^{j\alpha},$$

where Δ_j is the sawtooth function with 2^{j-1} teeth.

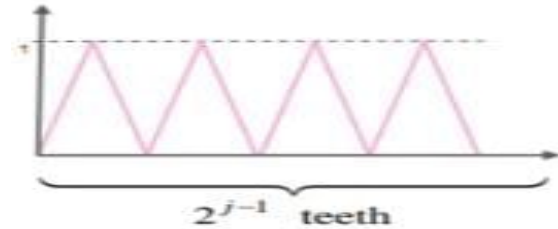


Figure 3: sawtooth function

פונקציית המסור מדרגה j ניתן לראות כמו בסקיצה בעלת 2^{j-1} שיניים.
אופרטור ההפרש מסדר שני של פונקצייה לינארית ב $\mathbb{R}^2$ הוא 0:

$$\begin{aligned} f(x) &= mx + n \\ \Delta_h &= mx + mh + n - mx - n = mh \\ \Delta_h^2 &= mh - mh = 0 \end{aligned}$$

עבור כל $t \leq 2^{-j-1}$:

$$\omega_2(\Delta_j, t)_\tau = \sup_{|n| \leq 2^{-j-1}} ||\Delta_h^2(\Delta_j, x)||$$

נגדיר $\Omega = [0, 2^{-j-1}]$, ולפי הגדרת מודולס החלקות שראינו בהצאה אם $[x, x + 2h] \not\subseteq \Omega$, אזי $\Delta_h^2(\Delta_j, x) = 0$.

הפונקצייה לינארית בקטע $[0, 2^{-j-1}]$ לכן כמו שהראנו שמתקיים לפונקצייה לינארית, $\omega_2(\Delta_j, t) = 0$ בקטע $[0, 2^{-j-1}]$.

ראינו גם שמתקיים $(**) (\omega_2(\Delta_j, t)_\tau \leq 2^2 ||\Delta_j||_{L^\tau} \leq 4$ קיבלנו אז,

$$|\Delta_j|_{B_\tau^\alpha[0,1]} = \left(\int_0^\infty \left(t^{-\alpha} \omega_2(\Delta_j, t)_\tau \right)^\tau \frac{dt}{t} \right)^{\frac{1}{\tau}} =$$

$$\begin{aligned}
& \underbrace{\left(\int_0^{2^{-j-1}} (t^{-\alpha}\omega_2(\Delta_j,t)_\tau)^\tau \frac{dt}{t}\right)^{\frac{1}{\tau}} + \left(\int_{2^{-j-1}}^\infty (t^{-\alpha}\omega_2(\Delta_j,t)_\tau)^\tau \frac{dt}{t}\right)^{\frac{1}{\tau}}}_{=0 \text{ לפי } (*)} \\
& \stackrel{(**)}{\leq} 4 \left(\int_{2^{-j-1}}^\infty (t^{-\alpha})^\tau \frac{dt}{t}\right)^{\frac{1}{\tau}} = C \left(\int_{2^{-j-1}}^\infty t^{-\alpha\tau-1} dt\right)^{\frac{1}{\tau}} = \left[C \frac{1}{-\alpha\tau} t^{-\alpha\tau}\right]^\tau \Big|_{2^{-j-1}}^\infty \\
& = 0 + \left(C \frac{1}{\alpha\tau}\right)^{\frac{1}{\tau}} (2^{-j-1})^{-\alpha} = C(\alpha,\tau)2^{-j\alpha}
\end{aligned}$$

וקיבלנו,

$$|\Delta_j|_{B^\alpha_\tau[0,1]} \leqslant C(\alpha,\tau)2^{j\alpha}$$

כנדרש.

5. [15%] Let $\{x_i, f(x_i)\}_{i \in I}$ be a dataset with $x_i \in [0, 1]^n$ and $f: [0, 1]^n \rightarrow \mathbb{R}^L$. Let $\mathcal{F}$ be a forest constructed over this data. For any $m > 0$, let $\tilde{x}_i = (x_i, z_i) \in [0, 1]^{n+m}$, $z_i \in \mathbb{R}^m$, $i \in I$ and $\tilde{f}: [0, 1]^{n+m} \rightarrow \mathbb{R}^L$, defined by $\tilde{f}(\tilde{x}) := f(\tilde{x}_1, \dots, \tilde{x}_n)$. Let $\tilde{\mathcal{F}}$ be the natural extension of $\mathcal{F}$ over $[0, 1]^{n+m}$ using the same trees with the same subdivisions over the first n dimensions. Prove that $N_\tau(\tilde{f}, \tilde{\mathcal{F}}) = N_\tau(f, \mathcal{F})$, for any $\tau > 0$.

$$f(x_i) = \tilde{f}(\tilde{x}_i) \text{ כִּי לראות כי}$$

בנוסף נתון כי z_i ללא השפעה ומתקבלים אותם עצים בדיוק.

ניזכר כעת בסימונים מההרצאה לשפה משותפת:

$$(I) \quad Q(x_i) = \frac{1}{\#\{x_i \in \Omega\}} \sum_{x_i \in \Omega} f(x_i) = \frac{1}{\#\{\tilde{x}_i \in \Omega\}} \sum_{\tilde{x}_i \in \Omega} \tilde{f}(\tilde{x}_i) = \tilde{Q}(\tilde{x}_i)$$

$$(II) \quad \psi(x_i) = 1_\Omega(x_i)(Q_{\Omega'}(x_i) - Q_\Omega(x_i)), \psi: [0, 1]^n \rightarrow \mathbb{R}^L$$

$$(III) \quad \tilde{\psi}(\tilde{x}_i) = 1_\Omega(\tilde{x}_i)(\tilde{Q}_{\Omega'}(\tilde{x}_i) - \tilde{Q}_\Omega(\tilde{x}_i)), \tilde{\psi}: [0, 1]^{n+m} \rightarrow \mathbb{R}^L$$

לכן, מתקבל מ II, III כי wavelets של כל אחד מה RF פונקצייה שונה, אולם אם נסתכל על I נקבל כי:

$$(IV) \quad \|\psi_{\Omega'}\|_2 = |\Omega'|^{1/2} \|Q_{\Omega'}(x) - Q_\Omega(x)\|_2 = |\Omega|^{1/2} \|\tilde{Q}_{\Omega'}(\tilde{x}) - \tilde{Q}_\Omega(\tilde{x})\|_2 = \|\tilde{\psi}_{\Omega'}\|_2$$

לכן קיבלנו מהגדרת $\mathcal{N}_\tau(f, \mathcal{F})$:

$$\mathcal{N}_\tau(f, \mathcal{F}) = \frac{1}{J} \left(\sum_{\Omega \in \mathcal{F}} \|\psi_\Omega\|_2^\tau \right)^{\frac{1}{\tau}} \stackrel{(IV)}{=} \frac{1}{J} \left(\sum_{\tilde{\Omega} \in \tilde{\mathcal{F}}} \|\tilde{\psi}_\Omega\|_2^\tau \right)^{\frac{1}{\tau}} = \mathcal{N}_\tau(\tilde{f}, \tilde{\mathcal{F}})$$

6) Show that for any $0 < \varepsilon < 1$, there exists a neural network $\tilde{D}$ with $O(\log^2(\varepsilon^{-1}))$ weights that approximates the function $D(x_1, x_2) = e^{x_1}x_2$, $\|D - \tilde{D}\|_{L_\infty[-1,1]^2} \leq \varepsilon$

יהי $0 < \varepsilon < 1$ ונתונות הרשתות D_1, D_2 המקיימות:

$$(I) \max_{-1 < x < 1} |e^x - D_1(x)| < C_1 e^{-n}$$

$$(II) \max_{-2e < x_1, x_2 < 2e} |x_1 x_2 - D_2(x_1, x_2)| < C_2 e^{-\sqrt{n}}$$

הרשתות הללו עם סדר גודל של n משקולות, ניתן לראות כי עבור $x_1 \in [-1, 1]$ מתקיים

$$\frac{1}{e} < e^{x_1} < e \implies -2e < e^{x_1} < 2e$$

$$\begin{aligned} (II) \implies \max_{\substack{-2e < x_1, x_2 < 2e \\ -1 < x_1 < 1}} |e^{x_1} x_2 - D_2(D_1(x_1), x_2)| &< C_2 e^{-\sqrt{n}} \end{aligned}$$

$$\text{for } n = \log^2\left(\frac{1}{\varepsilon} C_2\right)$$

$$\max_{\substack{-2e < x_1, x_2 < 2e \\ -1 < x_1 < 1}} |e^{x_1} x_2 - D_2(D_1(x_1), x_2)| \leq \varepsilon$$

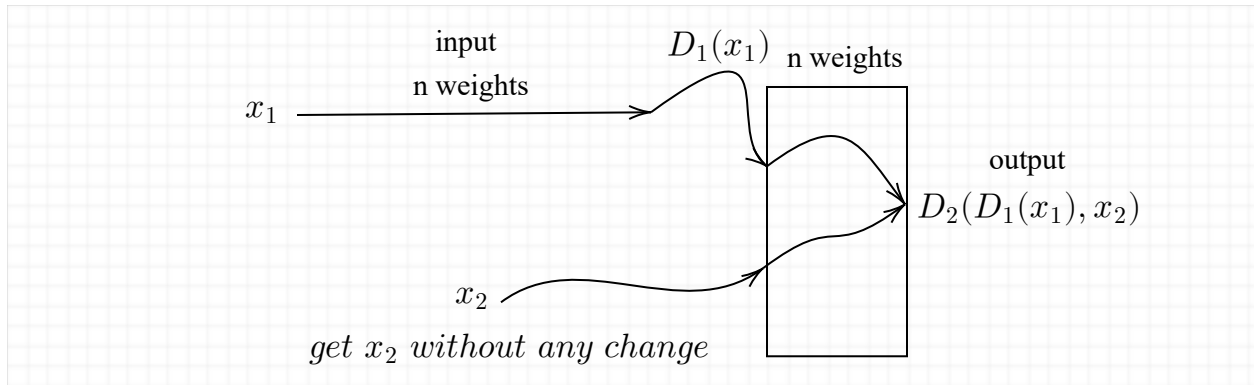
Big-O מסמן את סדר גודל החסם העליון על מספר המשקולות, מתקיים גם

$$\log(C_2 \varepsilon^{-n}) \leq \log^2(C_2 \varepsilon^{-n}), \forall 0 < \varepsilon < 1$$

נתבונן כעת ברשת החדשה $\tilde{D}$ אשר מוגדרת

$$\tilde{D}(x_1, x_2) = D_2(D_1(x_1), x_2)$$

רשת זאת בנוייה בדומה לסקיצה הבאה:



כמו שניתן לראות ברשת החדשה שבנינו מופעלות סה"כ $2n$ משקולות לכן,

$$\text{for } n = \log^2\left(\frac{1}{\varepsilon}C_2\right)$$

$$2n = 2\log^2\left(C_2\frac{1}{\varepsilon}\right) = 2\left(\log(C_2) + \log\left(\frac{1}{\varepsilon}\right)\right)^2 \Rightarrow O\left(\log^2\left(1/\varepsilon\right)\right)$$

so we get that,

$$\begin{aligned} \|D - \tilde{D}\|_{L_\infty[-1,1]^2} &= \text{esssup}\{D - \tilde{D}\} = \text{esssup}|D(x_1, x_2) - \tilde{D}(x_1, x_2)| = \\ &= \max_{\substack{-2e < x_1, x_2 < 2e \\ -1 < x_1 < 1}} |D(x_1, x_2) - \tilde{D}(x_1, x_2)| = \max_{\substack{-2e < x_1, x_2 < 2e \\ -1 < x_1 < 1}} |e^{x_1}x_2 - D_2(D_1(x_1), x_2)| \end{aligned}$$

$$< C_2 e^{-\sqrt{\log^2(\varepsilon^{-1}C_2)}} = C_2 \frac{1}{e^{\log(C_2\varepsilon^{-1})}} = C_2 \frac{\varepsilon}{C_2} = \varepsilon$$

כנדרש.

7) Design a NN that will solve the following computer vision problem: You are given a blurry real-life RGB image, and you need to unblur/sharpen it.

- Describe how to create a training dataset for this problem.
- Describe a candidate NN architecture and explain its logic.
- Describe what loss function(s) can be used during training.

סעיף א':

לצורך יצירת סט אימון למודל שיפתור את הבעיה הזאת נצטרך כמו שראינו בהרצאה בנושא super-resolution, ניקח תמונות חדות ← נטשטש אותם באמצעות נגיד Gaussian blur לערכי ה RGB ונזין את המודל בתמונות המטושטשות כדי שילמד לחדד אותן (חשוב לשים לב שבמידה והתמונות יטושטשו בצורה ששונה מאוד מאיך שהתמונות אימון טושטשו או גודל התמונה שונה מהתמונות שהתאמן עליהן - המודל ככל הנראה לא ידע להתמודד עם זה בצורה טובה, כמו שראינו בהרצאות מודל מעולה על הבעיה הספציפית שהוא לומד עליה).

דיוק המודל או כמה טוב הוא חידד את התמונות המטושטשות על-ידי השוואה לתמונות המקוריות, על ידי אדם שידרג את איכות החידוד או על-ידי מדד שיקבע על ידי המשתמש של השוואת התמונות (נגיד הפרש הערכים בין תמונת המקור לתמונת המודל).

סעיף ב':

אריטקטורה מועמדת לרשת כזאת, דומה לרשת שראינו בהרצאה עבור סופר-רזולוציה.

קלט הרשת: תמונת RGB מטושטשת $H \times W \times 3_{RGB}$

נבנה רשת CNN כאשר הקונבולוציות מחפשות מאפיינים מקומיים (כל איזור בתמונה בעל מאפיינים מיוחדים לו גם לאחר הטשטוש).

בס Hidden layers בשכבה לפני האחרונה, בגלל שהקונבולוציות משנות את מימדי התמונה נצרך לחזור לרזולוציה המקורית, נשתמש ברעיון sub-pixel convolution שראינו בהרצאה בנושא סופר רזולוציה-לכן בין היתר הקונבולוציות מחפשות מאפיינים מקומיים.

השכבה האחרונה בנויה על הרעיון של אינטרפולציה, כאשר לכל מיקום יש משקול שונה כמו שראינו בהרצאה על סוגים שונים של אינטרפולציה.

ואז הרשת לומדת לעבור מ L מימדים ל 3 מימדים.

פלט הרשת: מאפיינים של התמונה שהרשת למדה $g(H) \times h(W) \times L(3r^2)$

כאשר r הוא היפר פרמטר של "אגרסיביות" החידוד תמונה או unblur. מאחר ו r^2 הערך "מתקן" את התמונה

$g(H) \times h(W) \times L(3r^2)$ לגודל $H \times W \times 3_{RGB}$.

בעצם רשת זאת בנויה כמו שראינו בהרצאה על הרצה של התמונות ברשת קיימת כבר למטרה אחרת (לדוגמה סופר רזולוציה, VGG) ו"חיתוך" של הרשת כדי ויצירת המשך חדש שיתאים למשימה החדשה שלנו.

סעיף ג':

פונקציית המחיר נגדיר כשילוב ממושקל של MSE P2P loss ושל VGG loss

$$Loss = MSE_{all} + \lambda VGG$$

$VGG-loss: \sum_{j=j_1}^{j_2} \sum_{i=1}^n \left(\phi_j(\widehat{I}_i) - \phi_j(I_i) \right)^2$, where I_i is the original sharp image and
 $j_1 \leq j \leq j_2$ ϕ_j is the VGG layers where we want to check the features

$MSE \text{ of all images: } \frac{1}{nHW} \sum_{k=1}^n \sum_{i=1}^H \sum_{j=1}^W \left(\widehat{I}_k - I_k \right)^2$, where I_k is the original sharp image

so we get,

$$\begin{aligned}
 Loss &= MSE_{all} + \lambda VGG = \\
 &= \frac{1}{nHW} \sum_{k=1}^n \sum_{i=1}^H \sum_{j=1}^W \left(\widehat{I}_k - I_k \right)^2 + \lambda \sum_{j=j_1}^{j_2} \sum_{i=1}^n \left(\phi_j(\widehat{I}_i) - \phi_j(I_i) \right)^2
 \end{aligned}$$

8. [10%] Design a NN $\tilde{u}$ and a loss function for the following problem. You need to solve the 2D heat equation

$$\frac{\partial u}{\partial t}(x, t) = \frac{\partial^2 u}{\partial^2 x_1}(x, t) + \frac{\partial^2 u}{\partial^2 x_2}(x, t),$$

in the domain $x = (x_1, x_2) \in [0, 1]^2$, $t \in [0, 10]$, with initial conditions $u(x, 0) = f(x)$, and (compatible) boundary conditions

$$u((0, x_2), t) = g_1(t), u((1, x_2), t) = g_2(t), u((x_1, 0), t) = g_3(t), u((x_1, 1), t) = g_4(t), t \in [0, 10]$$

The input to the network is $x = (x_1, x_2) \in [0, 1]^2$, $t \in [0, 10]$, and the output is a scalar $\tilde{u}(x, t)$. Explain the role of automatic differentiation during training.

נסמן ב- $\tilde{u}$ כרשת הנוירונים.

נגדיר MSE

$$MSE_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |u((x_1, x_2), 0) - \tilde{u}((x_{1i}, x_{2i}), 0)|^2, N_0 \text{ num of samples in time } N_0 \text{ start point}$$

עבור ערך שהוא לא 0 או 1 בנקודה בגבול התחום אין משמעות על הערך בנקודה ולכן נוכל להגדיר קנס על נק כך:

$$MSE_{boundary} = \frac{1}{N_{(0,x_2)}} \sum_{i=1}^{N_{(0,x_2)}} |\tilde{u}((0, x_{2i}), t_i) - g_1(t_i)|^2 + \frac{1}{N_{(1,x_2)}} \sum_{i=1}^{N_{(1,x_2)}} |\tilde{u}((1, x_{2i}), t_i) - g_2(t_i)|^2 + \frac{1}{N_{(x_1,0)}} \sum_{i=1}^{N_{(x_1,0)}} |(\tilde{u}(x_{1i}, 0), t_i) - g_3(t_i)|^2 + \frac{1}{N_{(x_1,1)}} \sum_{i=1}^{N_{(x_1,1)}} |(\tilde{u}(x_{1i}, 1), t_i) - g_4(t_i)|^2$$

שכל חלק ב-MSE זה צלע אחרת בגבול.

נגדיר כעת לנקודות בתחום (משוואת החום 2D)

$$MSE_{inside} = \frac{1}{N_{in}} \sum_{i=1}^{N_{in}} \left(\frac{\partial \tilde{u}}{\partial t}(x_i, t_i) - \frac{\partial^2 \tilde{u}}{\partial^2 x_1}(x_i, t_i) - \frac{\partial^2 \tilde{u}}{\partial^2 x_2}(x_i, t_i) \right)^2$$

השימוש ב Automatic Diffrentiation(AT) אשר מאפשר למחשב לחשב נגזרות לפונקציות בצורה מהירה וזולה חישובית

יחסית, מתרחש לאחר שבתהליך האימון מחושב $\tilde{u}$ ו-MSE של הגבול ונקודות ההתחלה, בחישוב MSE_{inside} משתמשים ב

AT לקבלת הביטויים במשוואת החום

$$\frac{\partial \tilde{u}}{\partial t}; \frac{\partial^2 \tilde{u}}{\partial^2 x_1}; \frac{\partial^2 \tilde{u}}{\partial^2 x_2}$$

לכל נקודה, כך מתאפשר למודל לקבל דיוק טובה יותר למידול התנהגות הגלי חום ומציאת מקורם.

קלט הרשת: מטריצה בגודל $M_{n \times 4}$ כאשר אנחנו משתמשים ב 3 העמודות של זמן ומיקום במרחב באימון והעמודה הרביעית

משמשת למדד דיוק המודל. כאשר נכניס במודל הראשוני 50 נק מכל סוג בגבול ועוד 30000 בתוך התחום.

מבנה הרשת: הרשת בנויה מ-P שכבות קונבולוציה כדי למצוא משקולות שיקרבו בצורה הטובה ביותר למשוואת החום

המקורית.

הרשת היא Fully connected מתוך כך שכל הנקודות בכל נק זמן משפיעות ומושפעות אחת מהשנייה וזה יכול לעזור למודל

ללמוד את זה טוב יותר.

פלט הרשת: השכבה האחרונה תחזיר ערך $\tilde{u}(x_1, x_2, t) \in \mathbb{R}$.

9. [10%] You are given the problem of finding the point source location at time 0, of the 3D wave equation in the 3D domain $[0,1]^3$, given only samples of the solution at fixed time T

$$\left\{ u(x_i, T) : x_i = \left(\frac{i_1}{N}, \frac{i_2}{N}, \frac{i_3}{N} \right), 0 \leq i_1, i_2, i_3 \leq N \right\}.$$

- Describe how you would create synthetic dataset for this problem (to be split later to training and testing datasets).
- Provide a sketch design for the network architecture and write the loss function.

סעיף א':

רשת הנוירונים צריכה לשערך את נקודת המקור (w, y, z) של הגל. לצורך כך נניח כי יש לנו קוביץ יחידה עם סנסורים במרחקים שווים אחד מהשני ומיקום הוא כמתואר בשאלה ומהווה את $\{x_i\}_{i=1}^k$ לצורך ייצור הסט מידע המלאכותי - נבצע m סימולציות עד זמן T , מיקום החיישנים לא משתנה אלא רק נקודת המקור בכל סימולציה. כך, נקבל את סט המידע הבא

$$\left\{ \left(\underbrace{\{(u, x)_i\}_{i=1}^k}_\text{features}, \underbrace{(w, y, z)_j}_{\text{source point } j} \right) \right\}_{j=1}^m$$

מתקבל סוג של מפה/תמונה תלת מימדים שהיא Snapshot של כל סימולציה בזמן T -בדומה למה שראינו בכיתה לדו מימד. התקבלה פה בעיית רגרסיה בה רוצים לחזור נקודה על ידי תגובת החיישנים ומשוואת הגלים.

סעיף ב':

בהינתן שהרשת לומדת מתמונת מצב(snapshot) בזמן T ו"תהנדס לאחור" את תמונת המצב בזמן 0 על ידי שימוש במשוואת הגלים, נקבל נק אפס משוערת. פונקציית מחיר הבדוקת את טיב המודל בבעיה דומה לזו (בכיתה ראינו בדו מימד) וניתן להשתמש בה כאן גם כן. אופציה אחרת היא האפשרות הקלאסית של MSE

$$MSE = \frac{1}{m} \sum_{i=1}^m \|(\hat{w}, \hat{y}, \hat{z})_i - (w, y, z)_i\|_2^2$$

